in tools like *dd* for Linux.

Clone hello-world project: Clone the balena Node.js hello-world example from the GitHub repository at *https://github.com/balena-io-examples/balena-node-hello-world* using Git clone.

Hardware needed

Since this is a hello-world project with the balena ecosystem, we'll keep it simple and use minimal hardware. We will be hosting a basic Web page created in Node.js on the Raspberry Pi, but we can always add sensors, actuators and displays as per our requirements. For this project, we'll need:

- A Raspberry Pi board
- SD card
- Power supply

Setting up the fleet

Step 1: **Create a new fleet**

The first step is to create a fleet on the balena cloud dashboard after logging in.

Here, you need to name your fleet; for this example, we'll call it hello-world. You can also choose a default device type for this fleet, which will be Raspberry Pi 4. We can name the fleet type *Starter* for now. Click on 'Create new fleet' to finish the initial setup. Once a fleet is created, you can add devices to the fleet using the 'Add device' button.

Step 2: **Add a new device to the fleet**

This page will allow us to download balenaOS for our device. Here, we can also select the device you want to add. Raspberry Pi 4 is here by default from the fleet creation step, but you can change it. Choose the recommended version of balenaOS.

We also have the option to choose the balenaOS image for production that is more secure. The locked in version is meant to be deployed in production with ssh disabled, but we'll choose a development image for this demo since we're not deploying this device in the field yet. The good thing is that you can also choose how you want to connect the device to the Internet. You can choose only Ethernet or Ethernet + Wi-Fi. You can also add your Wi-Fi credentials on this page, so that the balenaOS image you download here comes baked with these credentials and the device automatically connects with the hotspot. This is quite handy for quick prototyping.

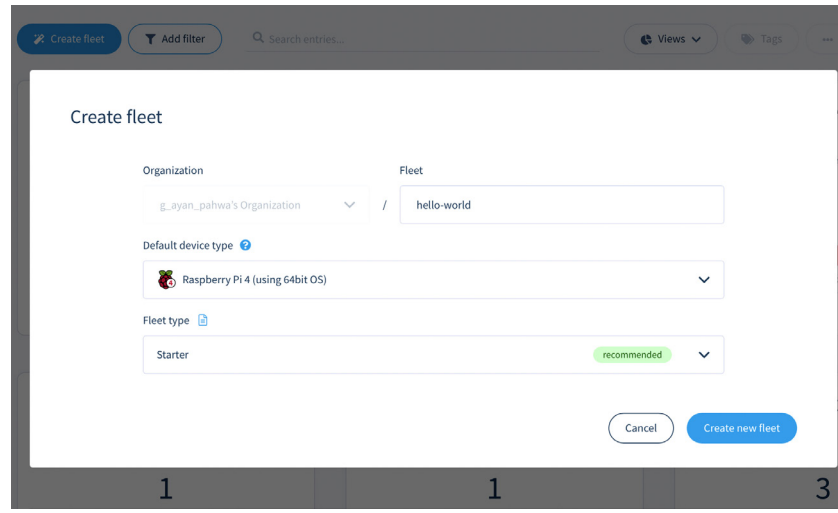

Figure 3: Creating a new fleet

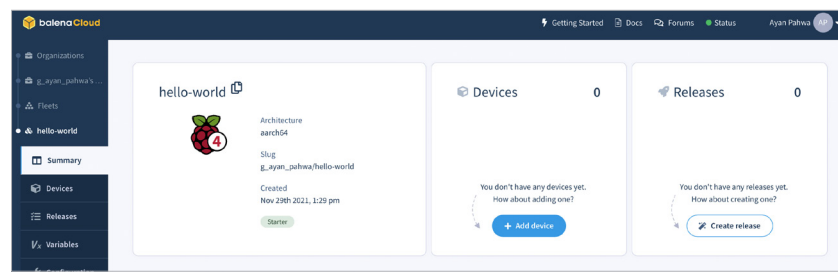

Figure 4: Fleet summary

Step 3: **Pushing code to the device(s)**

After powering on the Raspberry Pi, if you go to the device pane on the balenaCloud dashboard you'll be able to see your device online. We now need to push the code onto our device, for which we will use balenaCLI.

In the terminal (Linux/Mac) or balenaCLI.exe (Windows), execute the following command:

```
balena push <fleet name>
```

To associate your balenaCloud account with balenaCLI, you can choose any available login method. The preferred one is Web authorisation, which will authenticate using a Web browser. Finally, to push the code to the devices in the fleet, we'll use:

```
balena push hello-world
```

This will update the devices with the latest release.

How does this work?

Balena push checks for *dockerfile* or *docker-compose* file and *build* container images for your device architecture, and makes your balenaCloud dashboard available for updating all the devices in the fleet. The images are built on the balena infrastructure, but there's also a way to build images locally using something called local-mode.

Now let's verify if our app is working off the Raspberry Pi device. From the